

Sentinel Lab

Mini SOC • Informe Técnico

STACK TECNOLÓGICO

Wazuh

Suricata

Grafana

Snort

DefectDojo

Cortex

Sigma

OpenVAS

YARA

rsyslog

Este es el informe oficial sobre el proyecto **SentinelLab MiniSOC** realizado por **Ángel Castaño Arias**. Este busca reflejar los conocimientos adquiridos en el “**Certificado Optimización y Gestión de la Monitorización de Seguridad**” impartido en la plataforma OpenWebinars por el profesor **Sergio Palacios Domínguez**.

Si deseas consultar el curso aquí tienes el enlace: **Optimización y Gestión de la Monitorización de Seguridad**.

ÍNDICE DE CONTENIDOS

ÍNDICE DE CONTENIDOS.....	2
1. INTRODUCCIÓN.....	3
1.1 Objetivo del Proyecto.....	3
1.2 Tecnologías Empleadas.....	3
2. Arquitectura.....	4
2.1 Tabla de Máquinas virtuales.....	4
2.2 Arquitectura.....	4
2.3 Flujo de Logs.....	5
3. Bloque 1 — Fundamentos y Syslog.....	6
3.1 Objetivo del Bloque.....	6
3.2 Servicios Instalados en VM1.....	6
3.2.1 OpenSSH Server.....	6
3.2.1 Apache Server.....	7
3.3 Configuración de rsyslog.....	7
3.3.1 Fichero de Configuración Personalizado.....	7
3.3.2 Creación del Directorio de Logs.....	8
3.3.3 Estructura de Logs Resultante.....	8
3.4 Pruebas Realizadas y Resultados.....	8
3.4.1 Prueba 1 — Intentos SSH Fallidos.....	8
3.4.2 Prueba 2 — Accesos Web.....	9

1. INTRODUCCIÓN

1.1 Objetivo del Proyecto

Construir un entorno casero de monitorización de seguridad que recorra de forma práctica los 6 bloques del curso **Optimización y Gestión de la Monitorización de Seguridad**:

- Fundamentos de monitorización
- Detección en red y endpoint
- Correlación en SIEM
- Threat Intelligence
- Gestión de vulnerabilidades
- Detection as Code

1.2 Tecnologías Empleadas

Categoría	Herramienta
Sistema Operativo	Ubuntu Server 22.04 LTS
Logs	rsyslog
IDS Red	Snort, Suricata
IDS Endpoint	Wazuh Agent
Detección Malware	YARA
SIEM	Wazuh Server
Visualización	Grafana
Threat Intel	AbuseIPDB, OTX, ThreatFox
SOAR	Cortex
Vulnerabilidades	OpenVAS, DefectDojo
Atacante	Kali Linux

2. Arquitectura

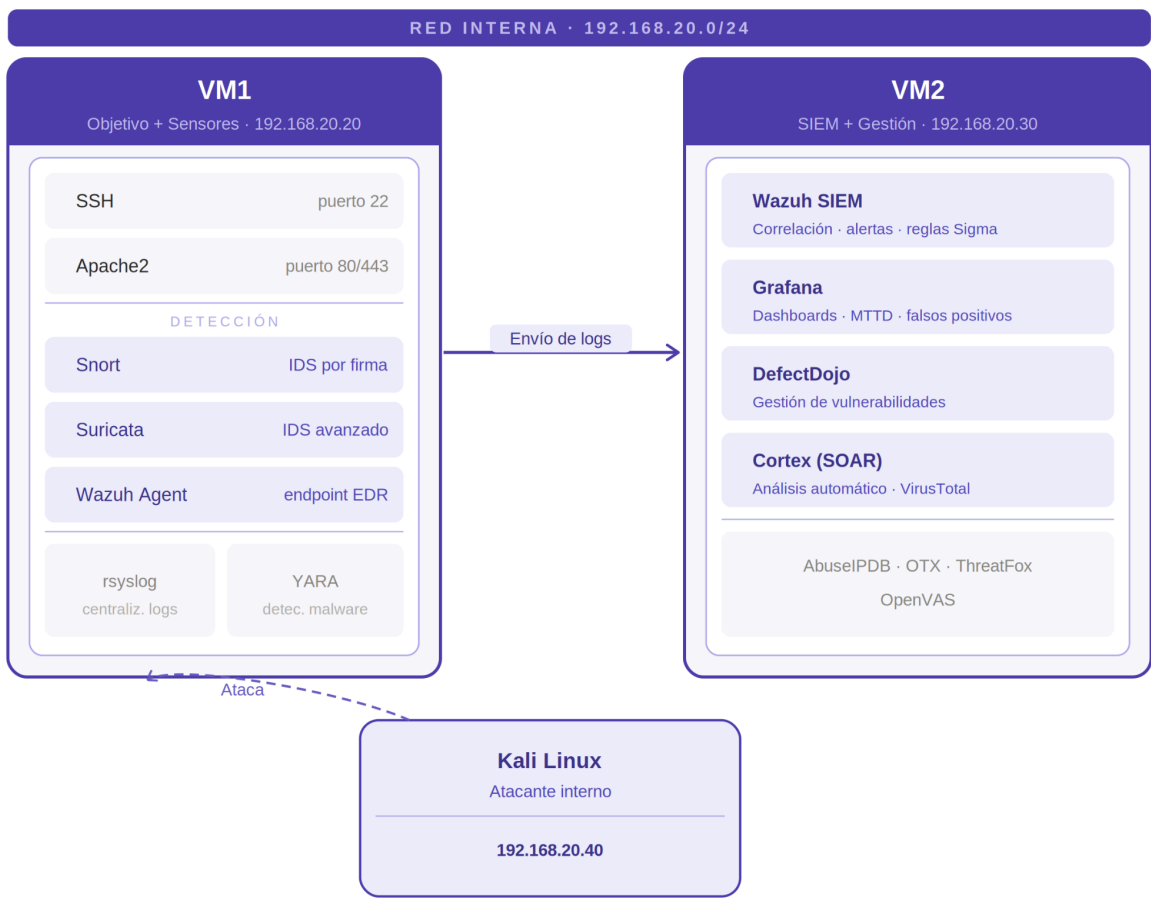
2.1 Tabla de Máquinas virtuales

El proyecto está diseñado con **solo 3 máquinas virtuales** para optimizar recursos, aquí información sobre las mismas:

Máquina	Rol	Sistema Operativo	IP
VM1	Objetivo + Sensores	Ubuntu Server 22.04	192.168.20.20
VM2	SIEM + Gestión	Ubuntu Server 22.04	192.168.20.30
Kali	Atacante externo	Kali Linux	DHCP (192.168.20.x)

2.2 Arquitectura

Así mismo la **arquitectura** establecida sobre estas máquinas virtuales es la siguiente:



2.3 Flujo de Logs

El flujo de logs establecido es el siguiente:



3. Bloque 1 – Fundamentos y Syslog

3.1 Objetivo del Bloque

Establecer la base del entorno de monitorización:

- Diseñar la arquitectura de red del mini-SOC
- Exponer servicios reales (SSH, HTTP) como superficie de ataque
- Configurar rsyslog para centralizar eventos críticos
- Contrastar monitorización pasiva vs activa

3.2 Servicios Instalados en VM1

3.2.1 OpenSSH Server

3.2.1 OpenSSH Server

Servidor SSH expuesto como vector de ataque simulado

PUERTO 22 / TCP	INSTALACIÓN <pre>sudo apt install openssh-server -y</pre> <pre>sudo systemctl enable ssh</pre> <pre>sudo systemctl start ssh</pre> VERIFICACIÓN <pre>sudo systemctl status ssh</pre>
FUNCIÓN Simular servidor SSH expuesto al atacante	
ESTADO enabled · active	

3.2.1 Apache Server

3.2.2 Apache Server

Servidor web expuesto como vector de ataque simulado

PUERTO

80 / TCP

FUNCIÓN

Simular servidor web
expuesto al atacante

ESTADO

enabled · active

INSTALACIÓN

```
sudo apt install apache2 -y
```

```
sudo systemctl enable apache2
```

```
sudo systemctl start apache2
```

VERIFICACIÓN DESDE KALI

```
curl http://192.168.20.20
```

3.3 Configuración de rsyslog

3.3.1 Fichero de Configuración Personalizado

Procedemos a la creación de ficheros personalizados para establecer las reglas de monitorización, para ello nos tenemos que dirigir a la ruta: **/etc/rsyslog.d/50-sentinellab.conf**.

En esta crearemos los siguientes apartados separando por tipo de regla, quedando algo tal que así:

```
/etc/rsyslog.d/50-sentinellab.conf *
# =====
# Sentinellab – Reglas de monitorización
# VM1 – Objetivo + Sensor
# =====

# 1. Intentos de login SSH y autenticación
auth,authpriv.*      /var/log/sentinellab/auth.log

# 2. Mensajes del kernel
kern.*                /var/log/sentinellab/kernel.log

# 3. Warnings o superior de cualquier servicio
*.warn                /var/log/sentinellab/warnings.log
```

3.3.2 Creación del Directorio de Logs

Posteriormente, para que se puedan almacenar los logs, crearemos el siguiente directorio, además de asignarle los permisos necesarios:

```
vm1@vm1:~$ sudo mkdir -p /var/log/sentinellab
vm1@vm1:~$ sudo chown syslog:adm /var/log/sentinellab
vm1@vm1:~$ sudo systemctl restart rsyslog
```

3.3.3 Estructura de Logs Resultante

Con estos cambios aplicados nos quedaría una estructura tal que así:

```
vm1@vm1:~$ tree /var/log/sentinellab/
/var/log/sentinellab/
├── auth.log
├── kernel.log
└── warnings.log

0 directories, 3 files
```

3.4 Pruebas Realizadas y Resultados

3.4.1 Prueba 1 — Intentos SSH Fallidos

Desde Kali Linux realizamos varios intentos de login fallidos a la VM1:

```
ssh usuariofalso@192.168.20.20
The authenticity of host '192.168.20.20 (192.168.20.20)' can't be established.
ED25519 key fingerprint is: SHA256:NP6h+ff653yaive00jBLTPV7hRsZpSLk7fk9uHqbKhE
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.20.20' (ED25519) to the list of known hosts.
usuariofalso@192.168.20.20's password:
Permission denied, please try again.
usuariofalso@192.168.20.20's password:
Permission denied, please try again.
usuariofalso@192.168.20.20's password:
usuariofalso@192.168.20.20: Permission denied (publickey,password).

ssh root@192.168.20.20
root@192.168.20.20's password:
Permission denied, please try again.
root@192.168.20.20's password:
Permission denied, please try again.
root@192.168.20.20's password:
root@192.168.20.20: Permission denied (publickey,password).
```

Dentro de la VM1, podremos ver reflejados los resultados de estos intentos en `/var/log/sentinellab/auth.log`, siendo estos algo tal que así:

```
vm1@vm1:~$ tail /var/log/sentinellab/auth.log
Jul 1 22:02:48 vm1 sudo: pam_unix(sudo:session): session closed for user root
Jul 1 22:10:29 vm1 sshd[3630]: Invalid user usuariofalso from 192.168.20.30 port 47002
Jul 1 22:10:32 vm1 sshd[3630]: pam_unix(sshd:auth): check pass; user unknown
Jul 1 22:10:32 vm1 sshd[3630]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=
0 tty=ssh ruser= rhost=192.168.20.30
Jul 1 22:10:35 vm1 sshd[3630]: Failed password for invalid user usuariofalso from 192.168.20.30
port 47002 ssh2
Jul 1 22:10:39 vm1 sshd[3630]: message repeated 2 times: [ Failed password for invalid user usua
riofalso from 192.168.20.30 port 47002 ssh2]
Jul 1 22:10:39 vm1 sshd[3630]: Connection closed by invalid user usuariofalso 192.168.20.30 port
47002 [preauth]
Jul 1 22:10:51 vm1 sshd[3632]: Failed password for root from 192.168.20.30 port 54188 ssh2
Jul 1 22:10:51 vm1 sshd[3632]: Failed password for root from 192.168.20.30 port 54188 ssh2
Jul 1 22:10:51 vm1 sshd[3632]: Connection closed by authenticating user root 192.168.20.30 port
54188 [preauth]
```

3.4.2 Prueba 2 – Accesos Web

Desde Kali Linux tratamos de tomar datos de la VM1 a través de los siguientes comandos:

```
curl http://192.168.20.20
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-
transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
        padding: 0px 0px 0px 0px;
      }
    </style>
  </head>
  <body>
    <h1>It works!</h1>
  </body>
</html>
```

```
curl http://192.168.20.20/paginaquenoexiste
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr/>
<address>Apache/2.4.52 (Ubuntu) Server at 192.168.20.20 Port 80</address>
</body></html>

curl http://192.168.20.20/admin
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr/>
<address>Apache/2.4.52 (Ubuntu) Server at 192.168.20.20 Port 80</address>
</body></html>
```

Dentro de la VM1, podremos ver reflejados los resultados de estos intentos en `/var/log/apache2/access.log`, siendo estos algo tal que así:

```
vm1@vm1:~$ tail /var/log/apache2/access.log
192.168.20.30 - - [01/Jul/2026:22:08:09 +0200] "GET / HTTP/1.1" 200 10926 "-" "curl/8.20.0"
192.168.20.30 - - [01/Jul/2026:22:08:41 +0200] "GET / HTTP/1.1" 200 10926 "-" "curl/8.20.0"
192.168.20.30 - - [01/Jul/2026:22:08:54 +0200] "GET /paginaquenoexiste HTTP/1.1" 404 436 "-" "curl/8.20.0"
192.168.20.30 - - [01/Jul/2026:22:09:01 +0200] "GET /admin HTTP/1.1" 404 436 "-" "curl/8.20.0"
```

4. Bloque 2 – Detección en Red y Endpoint

4.1 Objetivo del Bloque

Implementar un sistema de detección multicapa en VM1 que combine:

- **Detección por firma (Snort):** Identificar patrones conocidos de ataque
- **Detección avanzada (Suricata):** Análisis de protocolo y flujos
- **Detección de malware (YARA):** Escaneo de archivos en disco
- **Análisis de comportamiento (Wazuh Agent):** Monitorización del endpoint

Este bloque complementa la monitorización pasiva del Bloque 1 con detección activa en tiempo real.

4.2 Snort – Sistema de Detección de Intrusiones Basado en Firmas

4.2.1 Instalación y configuración

4.2.1 Snort — Instalación y configuración

Versión instalada: Snort 2.9.x.x

RED MONITORIZADA
192.168.20.0/24

MODO
IDS pasivo

CONFIG
/etc/snort/snort.conf

INSTALACIÓN

```
sudo apt install snort -y
```

RED MONITORIZADA

```
Home network: 192.168.20.0/24
```

VALIDACIÓN DE CONFIGURACIÓN

```
sudo snort -T -c /etc/snort/snort.conf
```

4.2.2 Reglas personalizadas creadas

Antes de crear las reglas personalizadas debemos de crear un directorio personalizado para nuestro proyecto:

```
vm1@vm1:/$ sudo mkdir -p /etc/snort/rules/sentinellab
```

Una vez que lo tenemos podemos crear el fichero *local.rules*.

```
GNU nano 6.2 /etc/snort/rules/sentinellab/local.rules *
# =====
# SentinelLab - Reglas Snort personalizadas
# =====
# Regla 1: Detectar intentos SSH desde Kali
alert tcp any any -> 192.168.20.20 22 (msg:"[SentinelLab] Intento de conexion SSH detectado"; sid:1000001; rev:1;)
# Regla 2: Detectar escaneo de puertos
alert tcp any any -> 192.168.20.20 any (flags:S; msg:"[SentinelLab] Posible escaneo SYN detectado"; sid:1000002; rev:1;)
# Regla 3: Detectar acceso HTTP
alert tcp any any -> 192.168.20.20 80 (msg:"[SentinelLab] Acceso HTTP detectado"; sid:1000003; rev:1;)
```

Estas reglas personalizadas debemos incluirlas en el archivo de configuración principal de Snort, concretamente en el **Step 7**.

```
GNU nano 6.2 /etc/snort/snort.conf *
#####
# Step #7: Customize your rule set
# For more information, see Snort Manual, Writing Snort Rules
#
# NOTE: All categories are enabled in this conf file
#####
# Note to Debian users: The rules preinstalled in the system
# can be *very* out of date. For more information please read
# the /usr/share/doc/snort-rules-default/README.Debian file
#
# If you install the official VRT Sourcefire rules please review this
# configuration file and re-enable (remove the comment in the first line) those
# rules files that are available in your system (in the /etc/snort/rules
# directory)
#
# site specific rules
include $RULE_PATH/local.rules
include /etc/snort/rules/sentinellab/local.rules
```